

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

Persistencia en BBDD (Room)



Universidad
Rey Juan Carlos

Persistencia en Android

Android utiliza un sistema de archivos similar a los sistemas de archivos basados en disco de otras plataformas.

El sistema ofrece varias opciones para que guardes los datos de tu aplicación:

Archivos específicos de la aplicación: almacena archivos destinados únicamente al uso de tu app. Ejemplos: archivos de datos estructurados (archivos JSON), archivos de texto sin formato, archivos multimedia.

Archivos compartidos: almacena archivos que tu app pretende compartir con otras apps, como archivos multimedia o documentos.

Preferencias: almacena datos sencillos en pares clave-valor (SharedPreferences).

Bases de datos: almacena datos estructurados en una base de datos privada para tu aplicación.

Bases de datos

¿Qué es una base de datos?

Una base de datos **es una colección de datos estructurados** a los que se puede acceder, buscar y organizar fácilmente. Los datos **se almacenan en tablas**, y cada tabla contiene campos de información relacionados.

Base de datos de ejemplo

Persona
_id
nombre
edad
dni

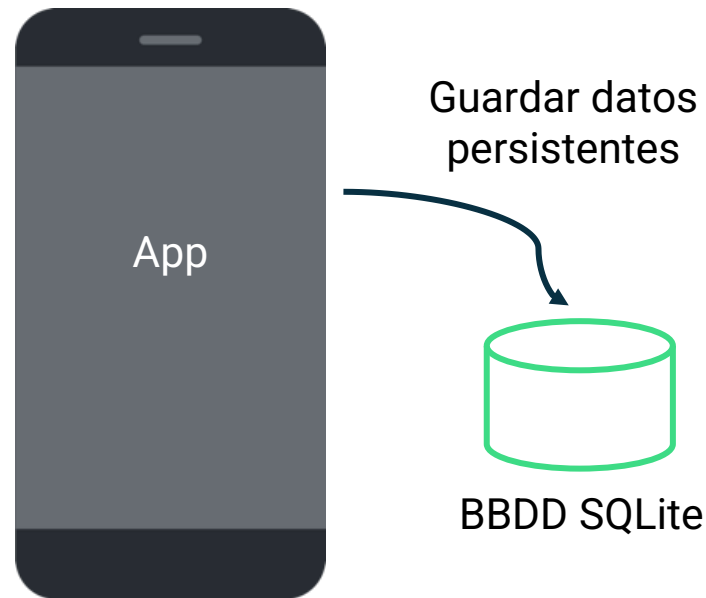
Vivienda
_id
referenciacatastral
superficie
habitaciones

Structured Query Language (SQL)

Para **interactuar con una base de datos relacional**, utilizamos SQL (o Lenguaje de Consulta Estructurado), que es un lenguaje específico para bases de datos.

En Android se utiliza SQLite, que está basado en el estándar SQL pero es más apropiado para dispositivos embebidos. Lo usaremos para:

- Crear nuevas tablas
- Consultas datos
- Insertar datos
- Actualizar datos
- Borrar datos



Ejemplos de comandos SQLite

Leer datos (consultar)

```
SELECT * from colors;
```

Insertar datos

```
INSERT INTO colors VALUES ("red", "#FF0000");
```

Actualizar datos

```
UPDATE colors SET hex="#DD0000" WHERE name="red";
```

Borrar datos

```
DELETE FROM colors WHERE name = "red";
```

En *View > Tool Windows > App Inspection* se puede interactuar con la base de datos de nuestra aplicación, inspeccionarla, hacer queries...

Librería Room

Añadir las dependencias

Para utilizar la librería *Room* hay que añadir las dependencias necesarias en los archivos *build.gradle.kts* del proyecto.

```
plugins {  
    id("org.jetbrains.kotlin.kapt") version "1.9.22" apply false  
    ...  
}
```

build.gradle.kts (Project)

```
plugins {  
    id("org.jetbrains.kotlin.kapt")  
}  
  
dependencies {  
    implementation("androidx.room:room-runtime:2.6.1")  
    kapt("androidx.room:room-compiler:2.6.1")  
    implementation("androidx.room:room-ktx:2.6.1")  
}
```

build.gradle.kts (Module)

¿Qué es Room?

Es una biblioteca de persistencia que brinda **una capa de abstracción para SQLite**, permitiendo acceder a la BBDD de forma simplificada y evitando errores.

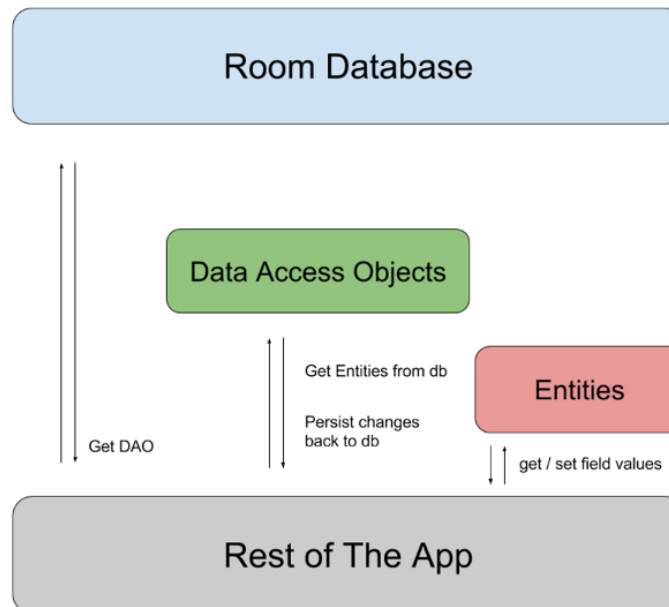
En particular, Room brinda los siguientes beneficios:

- Verificación en tiempo de compilación de las consultas en SQL
- Anotaciones que minimizan el código necesario
- Rutas de migración de bases de datos optimizadas

Componentes de Room

Hay tres componentes principales en Room:

- **Entidad:** representa una tabla dentro de la base de datos
- **DAO:** contiene los métodos utilizados para acceder a la base de datos
- **Base de datos:** contiene el soporte de la BBDD y es el principal punto de acceso a los datos relacionales persistentes de la aplicación



Anotaciones

Las anotaciones son notas que explican o resaltan palabras o pasajes de un texto.

En los lenguajes de programación, las anotaciones **adjuntan metadatos** al código y pueden **proporcionar información adicional al compilador**. En Java, es habitual ver la anotación `@Override` cuando se sobrescriben métodos de una superclase en una subclase.

Room utiliza esta información para **generar código automáticamente**.

Por ejemplo: `@Entity` marca las entidades, `@Database` las bases de datos., y `@Dao` los objetos de acceso a las BBDD

Entity

Las entidades son **clases que se mapean a tablas SQLite**. Se implementan generalmente como clases de datos (***data class***).

La clase se anota como `@Entity`, indicando el nombre de la tabla. Cada atributo de la clase representa una columna de la tabla. El anotado como `@PrimaryKey` será único y puede autogenerarse. El resto se anotan como `@ColumnInfo` si queremos que el nombre del atributo y el de la columna en la tabla difieran.

```
@Entity(tableName = "pelicula_table")
data class Pelicula (
    @PrimaryKey(autoGenerate = true)
    val peliculaId: Int?,
    @ColumnInfo(name = "pelicula_titulo")
    val titulo: String,
    @ColumnInfo(name = "pelicula_anyo")
    val anyo: Int,
    val director: String
)
```

Pelicula.kt

pelicula_table
peliculaId
pelicula_titulo
pelicula_anyo
director

Data Access Object (DAO)

Para **acceder a los datos** de tu aplicación usando la librería de persistencia Room, trabajas con objetos de acceso a datos, o DAOs.

Una interfaz DAO define las interacciones deseadas con la base de datos, en lugar de utilizar constructores de consultas o consultas directas.

Trabajar con clases DAO en lugar de acceder directamente a la base de datos presenta importantes ventajas:

- Room crea la implementación del DAO en tiempo de compilación.
- Room verifica todas sus consultas DAO en tiempo de compilación.

Implementar un DAO

Anotamos la interfaz con **@Dao** para que el compilador genere la implementación automáticamente

```
@Dao
interface PeliculaDAO {
    @Insert
    fun insert(vararg pelicula: Pelicula)
```

vararg permite insertar más de un dato en una sola transacción

PeliculaDAO.kt

```
    @Update
    fun update(pelicula: Pelicula)
```

```
    @Delete
    fun delete(pelicula: Pelicula)
```

```
    @Query("SELECT * FROM pelicula_table")
    fun getAll(): LiveData<List<Pelicula>>
```

Anotamos cada método con *@Insert*, *@Update*, *@Delete* o *@Query* según corresponda

Para las consultas, los parámetros se inyectan con el formato :param

```
    @Query("SELECT * FROM pelicula_table WHERE pelicula_titulo = :nombre")
    fun getByName(nombre: String): LiveData<List<Pelicula>>
```

```
}
```

Crear una base de datos Room

Declaramos una clase abstracta que herede de *RoomDatabase*

Anotamos la clase con **@Database** e indicamos sus entidades

PeliculasRoomDatabase.kt

```
@Database(entities = [Pelicula::class], version = 1)
abstract class PeliculasRoomDatabase : RoomDatabase(){
    abstract fun peliculaDAO(): PeliculaDAO

    companion object{
        @Volatile
        private var INSTANCE: PeliculasRoomDatabase? = null

        fun getInstance(context: Context): PeliculasRoomDatabase{
            ...
        }
    }
}
```

Declaramos un método abstracto sin argumentos que devuelve el DAO

Declaramos un *singleton* que nos dé una instancia de la BBDD

@Volatile hace que la variable nunca se almacene en caché (siempre en RAM) por lo que los cambios siempre son visibles para todos los hilos

Crear la instancia de la base de datos

Usamos `Room.databaseBuilder()` para crear una base de datos usando el contexto de la aplicación, la clase de la BBDD y el nombre de la base de datos.

```
fun getInstance(context: Context): PeliculasRoomDatabase{
    val tempInstance = INSTANCE
    if(tempInstance != null){
        return tempInstance
    }
    synchronized(this){
        val instance = Room.databaseBuilder(
            context.applicationContext,
            PeliculasRoomDatabase::class.java, "pelicula_database"
        ).allowMainThreadQueries().build()
        INSTANCE = instance
        return instance
    }
}
```

PeliculasRoomDatabase.kt

`@synchronized` garantiza que solo un hilo entra en este bloque a la vez, asegurando que únicamente exista una instancia de la BBDD

`allowMainThreadQueries` NO SE UTILIZA. Solo para pruebas

Obtener y usar el DAO

Para conectarnos con la base de datos y hacer peticiones debemos:

1. Crear una instancia del objeto DAO correspondiente.
2. Utilizar alguno de sus métodos públicos para consultar, insertar, actualizar o borrar registros.

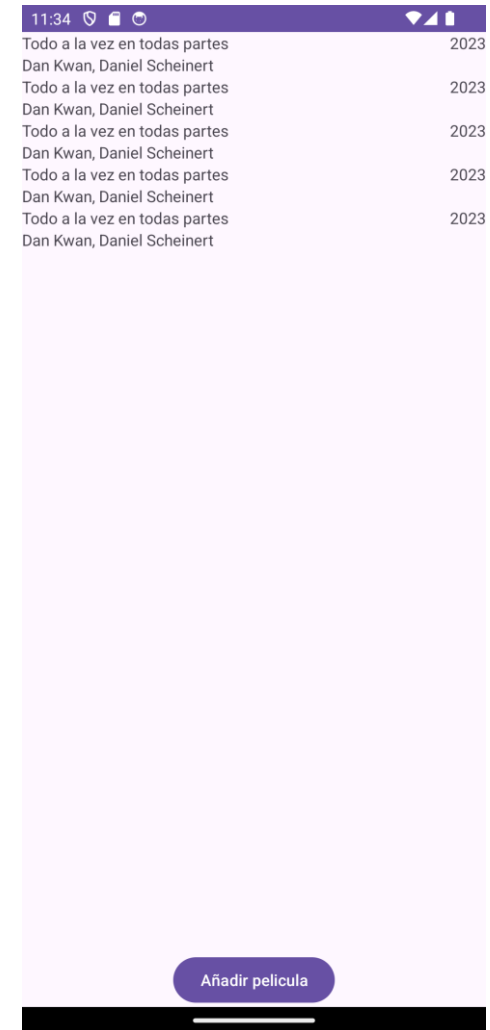
```
val películasDAO = PelículasRoomDatabase.getInstance(application).películaDAO()  
películas = películasDAO.getAll()
```

Ejercicio 1

Utilizando las implementaciones de Entidad, DAO y RoomDatabase de las diapositivas anteriores:

- Crea una aplicación con una única actividad que muestre una lista de películas en un RecyclerView tal y como se muestra en la captura de la derecha.
- El botón inferior debe añadir una película nueva (una predefinida, siempre la misma)
- Al cerrar la aplicación por completo y volverla a abrir, se deben mostrar las películas anteriormente añadidas.

Importante: para simplificar el ejercicio, la lista de películas observable se declarará en la *MainActivity*, prescindiendo así del *ViewModel*.



Crear ViewModel con acceso a la BBDD Room

En lugar de tener la BBDD en la Actividad, se modifica el VM para que tenga acceso a la BBDD Room a través de su DAO. Para ello, el VM necesita un *Context*.

```
class PeliculasViewModel(context: Context): ViewModel() {  
    private val myDAO: PeliculaDAO // DAO para interactuar con la BBDD  
    val peliculas: LiveData<List<Pelicula>> // Lista observable pública expuesta a la Actividad  
    init {  
        myDAO = PeliculasRoomDatabase.getInstance(context).peliculaDAO() // Inicializar DAO  
        peliculas = myDAO.getAll() // Asociamos la lista con la tabla de Peliculas de la BBDD  
    }  
  
    fun addPelicula(pelicula: Pelicula){  
        myDAO.insert(pelicula)  
    }  
  
    fun delPelicula(pelicula: Pelicula){  
        myDAO.delete(pelicula)  
    }  
}
```

PeliculasViewModel.kt

ViewModelFactory para VMs con parámetros

Para poder instanciar un ViewModel que recibe parámetros en su constructor, hay que crear una clase que herede de *Factory*, y devolver el VM en *create()*:

```
// Clase Factory para instanciar el ViewModel
// Recibe el mismo parámetro que el ViewModel y se lo pasa al crearlo
class PeliculasViewModelFactory(private val context: Context) : ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        return PeliculasViewModel(context) as T
    }
}
```

PeliculasViewModelFactory.kt

Para utilizar el VM se llama a *ViewModelProvider*, que nos permite construirlo a través de la factoría creada anteriormente, pasándole los parámetros adecuados:

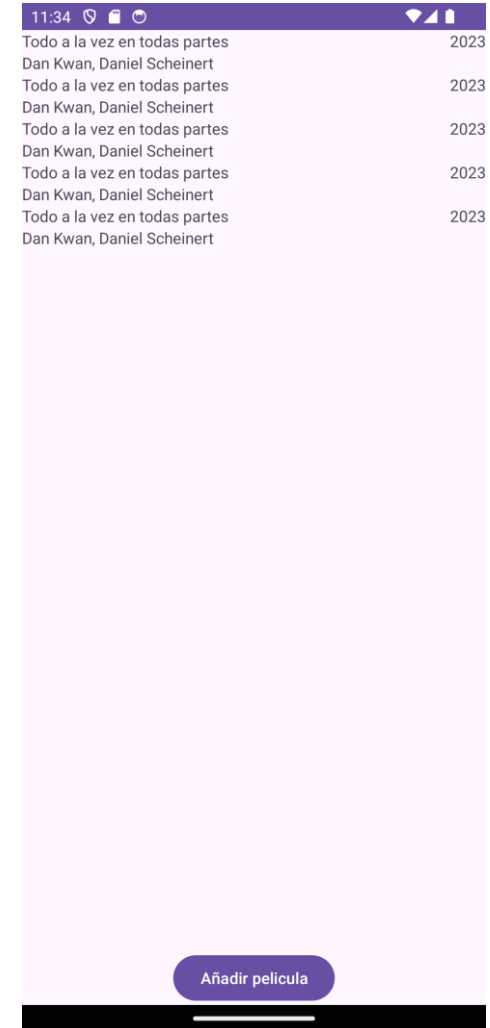
```
var viewModel = ViewModelProvider(
    this,
    PeliculasViewModelFactory(applicationContext)
)[PeliculasViewModel::class.java]
```

Ejercicio 2

Adapta la solución implementada en el ejercicio anterior para incluir un ViewModel que haga de intermediario entre la interfaz (Activity) y los datos (BBDD Room).

Para ello:

- Crea el ViewModel con conexión a una BBDD
- Crea la Factory asociada
- Adapta la actividad para que el RecyclerView se actualice cuando cambie la lista de películas del ViewModel



Ejercicio 3

Desarrolla una aplicación en Android para gestionar una biblioteca personal de libros, utilizando una base de datos local con Room.

La aplicación debe mostrar los libros en cuadrícula, utilizando un RecyclerView. Los libros tienen que estar almacenados en una base de datos que se accederá de forma segura a través de un ViewModel.

Para ello:

- La actividad principal mostrará los libros guardados en Room. Cada libro se verá con una carátula (siempre la misma imagen) y su título debajo
- El evento *onClick* en un libro nos llevará a una actividad donde se muestren todos los detalles del mismo
- El evento *onLongClick* borrará el libro de la base de datos
- Una actividad secundaria permitirá añadir libros nuevos a la base de datos

Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>